

From Data Projects to Pipelines

30th of May 2026 @ Dataharvest

Natalie Widmann

Agenda

- What are data pipelines and when to actually use them?
- Core Concepts: Tasks, Flows, and Schedulers
- Hands-on: building and running a pipeline with Prefect
- Best Practices

Demo - Party donations

What are data pipelines and when to actually use them?

Why not just run a script manually?

Some data collection starts with one person running a script on their laptop every morning. This works — until it doesn't.

Problems

- inconsistent timing and data gaps
- dependency on one person and their setup
- silent failures

What are data pipelines?

A data pipeline is an **automated, scheduled, and monitored workflow** for collecting, processing, or transforming data repeatedly over time.

When it's needed:

- Data Collection over a longer period of time (no historical data is available)
- Live dashboards
- Monitoring & Alerting
- Processing Tasks

Examples:

- Social media posts
- Water levels of rivers
- Air quality
- Donations to parties
- Heat days
- News classification

Demo

Forest Fire Alerts & Fuel Prices

Tools

- [Apache Airflow](#) - built by Airbnb in 2014, now developed by the Apache Software Foundation, large-scale data orchestration
- [Prefect](#) - modern Python framework, designed to fix issues with Airflow, easy to convert existing scripts
- [Luigi](#) - built by Spotify, lightweight Python jobs
- [n8n](#) - open-source workflow automation tool - focusses on connecting different apps and data integration, visual drag-and-drop (low code)

Basic Concepts

- **Task**

The **atomic unit of work**. One function that does one thing: fetch a file, clean a dataset, send a notification.

- **Flow / DAG / Pipeline**

A description of the full pipeline: which tasks run, in what order, and how they depend on each other. If one task fails the ones following are skipped.

Basic Concepts

- **Scheduler** runs the flow / pipeline automatically on a defined schedule.

Most tools use **cron syntax** for this:

```
*/5 * * * *    → every 5 minutes  
0 6 * * *      → every day at 6:00 AM
```

Also event-triggered, e.g. "run the script when a new file arrives", is an option.

Deployment

A **deployment** is the server-side configuration of a flow. It contains:

- **Where the code lives** (local path, Git repo, S3 bucket, etc.)
- **The schedule** (cron expression or event trigger)
- **The infrastructure** (which machine or container runs it)

The key benefit: once a deployment is defined, it's easy to switch from running on the laptop to running it on a server.

Demo Airflow

Hands-on: Prefect

Setup

1. Clone the Project

Open your terminal and run the following commands to download the repository and navigate into the project directory:

```
# Clone the repository
git clone https://github.com/Tilana/dataharvest2026_automated_pipelines

# Move into the project directory
cd dataharvest2026_automated_pipelines
```

2. Install uv (if you haven't already)

`uv` is an ultra-fast Python package and project manager. Install it using the official command for your operating system:

macOS & Linux

Run the standalone installer script via `curl`:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Alternative with Homebrew: `brew install uv`

3. Install Dependencies

Run `uv sync` in the root of the project directory to install all dependencies from the `pyproject.toml`

Run `uv run prefect --version` to check if the installation was successful.

Convert a script into a flow and tasks

```
from prefect import flow, task

@task
def fetch_data():

@flow
def main_workflow():
    data = fetch_data()
```

Humanfriendly Naming and Metadata

```
@task(name="Fetch Data", description="Fetches data from an API")
```

Deployment and Scheduling, e.g. run once every minute

```
if __name__ == "__main__":  
    main.serve(name="Dataharvest",  
               tags=["test"],  
               cron="* * * * *",  
               )
```

Logging

```
@flow(log_prints=True)
```

See how long tasks takd

```
@task(log_prints=True)
def fetch_data():
    ...
    time.sleep(3)
```

Graceful Failure

```
if number > 30:  
    raise ValueError("The number is too high!")
```

Retry

```
@flow(retries=2, retry_delay_seconds=2, log_prints=True)
def main_workflow():
    ...
```


Subflows

```
@flow
def subflow():
    ...

@flow
def main_workflow():
    ...
```

Parameters

```
def main(max_value=100):  
    ...
```

```
main.serve(  
    name="Dataharvest",  
    tags=["test"],  
    cron="* * * * *",  
    parameters={"max_value": 20},  
)
```

A file with all changes applied is given with `prefect_script.py` .

Run it with `uv run python prefect_script.py`

Deployment

Workflow + the configuration of the infrastructure
server-side representation of a flow which contains the configuration of the infrastructure

Instructions on how to run a flow, e.g. where is the code, what are the conditions and what is the schedule?

Allows you to easily switch infrastructure from local machine to a server.

Best Practices

Logging and Error Handling

- Assert data is available and correctly processed

Data Strategies

- store raw **and** processed data
- back up your data
- differentiate between a historical dump and new data coming in

Best Practices

Use consistent Data Formatting

- Naming Conventions
- Store the retrieval date
- Handle time zones explicitly

Use Template Scripts

Keep data catalogue

Resources

Tools

- [Apache Airflow](#) - built by Airbnb in 2014, now developed by the Apache Software Foundation
- [Prefect](#)
- [Luigi](#)
- [Crontab Guru](#)